

Labo 04 — Questions

Question 1 [Diagnostic]

Un développeur lance `podman build -t api:1.0 .` depuis `~/projets/api/`, et son Dockerfile contient `COPY ../commun/config.yml /app/`. Le build échoue avec `possible escaping context directory error`. Expliquez pourquoi, et dites pourquoi ni `-f`, ni un chemin absolu, ni `sudo` n'y changeront quoi que ce soit. Quelle est la solution correcte ?

Question 2 [Analyse]

Sous Docker, le build d'un projet Angular prend 4 minutes, dont 3 min 20 s affichées comme `transferring context`. Le dossier contient `node_modules/` (900 Mo) et `.git/` (200 Mo). Un collègue passe à Podman : le build ne prend plus que 40 secondes, et il en conclut que `.dockerignore` est devenu inutile. Expliquez ce qui se passait sous Docker, ce qui a changé sous Podman, et pourquoi il a tort — en nommant le **second** risque, indépendant de la lenteur.

Question 3 [Compréhension]

Un Dockerfile contient `EXPOSE 8080`. Le développeur lance `podman run -d mon-api:1.0` puis constate que `curl http://localhost:8080` ne répond pas. Il en conclut que l'image est cassée. A-t-il raison ? Expliquez le rôle exact d'`EXPOSE`.

Question 4 [Analyse]

Comparez ces deux Dockerfiles. Que fait précisément chacun, et lequel est correct pour une image d'API ?

```
# A
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
RUN java -jar /app/api.jar
```

```
# B
FROM docker.io/library/eclipse-temurin:21-jre
COPY api.jar /app/api.jar
CMD ["java", "-jar", "/app/api.jar"]
```

Question 5 [Analyse]

Voici deux images. Pour chacune, dites ce que produisent les commandes `podman run img` et `podman run img --debug`.

```
# A
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
# B
CMD ["java", "-jar", "/app/api.jar", "--spring.profiles.active=prod"]
```

Puis dites laquelle des deux permet encore de faire `podman run img sh` pour déboguer, et comment s'en sortir avec l'autre.

Question 6 [Diagnostic]

Une équipe se plaint que ses redéploiements durent toujours dix secondes de plus que prévu, que Podman affiche à chaque fois `resorting to SIGKILL`, et que Spring Boot n'exécute jamais ses *shutdown hooks*. Le Dockerfile se termine par :

```
CMD java -jar /app/api.jar
```

Diagnostiquez, corrigez, et expliquez pourquoi cette seule ligne suffit à produire le symptôme — et pourquoi l'équipe ne l'observe **pas** sur son image de test basée sur Alpine.

Question 7 [Analyse]

Un Dockerfile a besoin de la variable `JAVA_OPTS` au démarrage :

```
ENV JAVA_OPTS="-Xmx512m"  
ENTRYPOINT ["java", "$JAVA_OPTS", "-jar", "/app/api.jar"]
```

Le conteneur échoue avec `Unrecognized option: $JAVA_OPTS`. Expliquez, puis donnez **deux** corrections possibles en indiquant ce que chacune coûte.

Question 8 [Compréhension]

Un développeur passe le mot de passe de la base au build : `podman build --build-arg DB_PASSWORD=Secr3t! -t api:1.0 .`, en expliquant qu'un `ARG` ne persiste pas dans l'image. Est-il en sécurité ? Justifiez, et donnez la commande qui prouve votre réponse.

Question 9 [Analyse]

Un Dockerfile Maven est écrit ainsi :

```
COPY . /app  
WORKDIR /app  
RUN mvn -q package -DskipTests
```

Chaque build met 6 minutes, même quand un seul fichier `.java` a changé. Réécrivez les instructions dans le bon ordre, expliquez le mécanisme du cache qui rend votre version plus rapide, et dites quel build restera lent malgré tout.

Question 10 [Diagnostic]

```
RUN apt-get update  
RUN apt-get install -y curl vim  
RUN rm -rf /var/lib/apt/lists/*
```

Citez **trois** défauts distincts de ces trois lignes, et donnez la version correcte.

Question 11 [Analyse]

Après avoir modifié uniquement le dernier `COPY` de son Dockerfile, un développeur observe que le build affiche `--> Using cache` sur les huit premières étapes puis reconstruit les deux dernières. Le

lendemain, il ajoute une variable `ENV` en **troisième** position et le build entier repart de zéro. Expliquez les deux comportements avec la même règle.

Question 12 [Compréhension]

`COPY` et `ADD` semblent équivalents. Donnez deux comportements propres à `ADD`, expliquez pourquoi la recommandation officielle est d'utiliser `COPY`, et citez le seul cas où `ADD` reste justifié.

Question 13 [Analyse]

Un Dockerfile contient `USER 1000:1000` juste après le `FROM`, avant les instructions `COPY` et `RUN apt-get install`. Le build échoue en `Permission denied`. Expliquez, et dites où placer `USER` dans un Dockerfile bien écrit. Puis : en rootless, `podman top` montre pour ce conteneur `USER 1000` et `HUSER 100999`. Pourquoi ce second nombre, et l'instruction `USER` est-elle encore utile puisque « root » n'est déjà pas root ?

Question 14 [Analyse]

Votre collègue affirme : « Il faut réduire au maximum le nombre de couches, donc tout mettre dans un seul `RUN`. » Discutez : dans quels cas a-t-il raison, dans quels cas cette règle nuit-elle au temps de build et au poids des transferts ?